ASSESSMENT COVERSHEET

Attach this coversheet as the cover of your submission. All sections must be completed.

Section A: Submission Details

Programme	:	BACHELOR OF INFORMATION TECHNOLOGY (HONS) IN SOFTWARE ENGINEERING							
Course Code & Name	:	ISB37804 – REUSE AND COMPONENT BASED DEVELOPMENT							
Course Lecturer(s)	:	DR. AZIZAH RAHMAT							
Submission Title	:	PROJECT REPORT							
Deadline	:	Day	14	Month	1	Year	2026	Time	11:59 PM
Penalties	:	• 5% will be deducted per day to a maximum of four (4) working days, after which the submission will **not** be accepted. • Plagiarised work is an Academic Offence in University Rules & Regulations and will be penalised accordingly.							

Section B: Academic Integrity

Tick (√) each box below if you agree:

/	I have read and understood the UniKL's policy on Plagiarism in University Rules & Regulations.
/	This submission is my own, unless indicated with proper referencing.
/	This submission has not been previously submitted or published.
/	This submission follows the requirements stated in the course.

Section C: Submission Receipt

Office Receipt of Submission

Date & Time of Submission (stamp)	Student Name(s)	Student ID(s)
14/1/2026 11:59 PM	1. KHAIRUL AQIF DANIAL BIN KHAIRUL NIZAM 2. AKMAL HAKIMI BIN ABD RASHID 3. NUR HUSNA BATRISYA BINTI MOHD ZUHAILI	1. 52213123232 2. 52213123466 3. 52213123224

Student Receipt of Submission

This is your submission receipt, the only accepted evidence that you have submitted your work. After this is stamped by the appointed staff & filled in, cut along the dotted lines above & retain this for your record.

Date & Time of Submission (stamp)	Course Code	Submission Title	Student ID(s) & Signature(s)
14/1/2026 11:59 PM	ISB37804	PROJECT REPORT	1. 52213123232 2. 52213123466 3. 52213123224

Table of Contents

1.1 Background of the System 3
1.2 Problem Statement 4
2.0 Designing the Application - Scenario 5
2.1 Gathering User Requirements 5
Use Case Diagram 6
Component diagram 6
Package Diagram 7
Deployment diagram 7
Class diagram 8
2.2 Preparing Mockup 9
2.3 The Business Process Flow 15
2.4 Designing the Data Model 16
ER Diagram 16
Data Model 16
3.0 Future Recommendation/Suggestion 17
4.0 Conclusion 18
5.0 Appendix – Source Code Sample 19
6.0 References 25

Figure 1 Use Case Diagram 6
Figure 2 Component Diagram 6
Figure 3 Package Diagram 7
Figure 4 Deployment Diagram 7
Figure 5 Class Diagram 8
Figure 6 Login Page Mockup 9
Figure 7 Registration Page Mockup 9
Figure 8 Homepage Mockup 10
Figure 9 Menu Page Mockup 11
Figure 10 Booking Page Mockup 11
Figure 11 Reservation Details Mockup 12
Figure 12Booking Summary Mockup 13
Figure 13 Add New Item Page Mockup 13
Figure 14 Booking Management Mockup 14
Figure 15 Business Process Flow 15
Figure 16 ER Diagram 16

1.0 Introduction

LuxuryCarRental is an enterprise web application designed to meet the demands of the Reuse and Component, based Development course. The project is aimed at creating and implementing a solid system that makes use of standard enterprise components to address the real, world business problems of the car rental industry.

1.1 Background of the System

The LuxuryCarRental system is a web, based application that aims to digitize and oversee the core functions of a luxury vehicle rental service. The main goal of the system is to make the rental process of high, end vehicles easy by allowing customers to access an exclusive fleet through a secure online platform.

The system employs a component, based architecture, leveraging the Java Enterprise Edition (Java EE) technology stack to achieve modularity, reusability, and scalability. The development is in line with a multi, tier architectural pattern:

- Client Side (Front, end): The user interface is constructed with Java Server Pages (JSP) and CSS, thus it is an interactive web environment in which users can view vehicles and make bookings.
- Server Side (Back, end): The application logic is executed by Servlets and Enterprise JavaBeans (EJB). These elements interact with the client interface and the database through the components invoking the communication thus the data processing is efficient and there is a reuse of business logic.
- Database (Persistence): The system employs JavaDB (Apache Derby) as the relational database management system. Data persistence is handled by Java Persistence API (JPA), which maps Java objects to database tables, thus supporting the storage of user profiles, vehicle inventories, and booking records.

Integrating these components, the system provides functionalities for Customers to register, see available cars, and make reservations. At the same time, Administrators can maintain the fleet and supervise rental approvals. This method is in line with the course learning outcome to design and implement an enterprise application through reused and component, based development.

1.2 Problem Statement

Current manual or file, based car rental processes have to deal with a number of major problems:

1. **Availability Conflicts:** In the absence of real-time validation, the same vehicle can be double booked for the same dates, resulting in customer dissatisfaction.
2. **Inefficient Fleet Management:** Administrators have a hard time figuring out the status of a vehicle (Available vs. Unavailable) when they are using spreadsheets or paper records.
3. **Calculation Errors:** The manual calculation of rental costs based on daily rates and duration can easily make mistakes.
4. **Lack of Transparency:** Customers do not get to see their booking history or even the status (Pending, Approved, Rejected) in real, time.

2.0 Designing the Application - Scenario

2.1 Gathering User Requirements

Scope of the System

The LuxuryCarRental system is designed to support two distinct user roles (Actors), each with specific access privileges defined by the application's security logic:

1. **Customer:**
    - **Registration & Access:** Any user from the public can sign up for a new account. After registration, they can log in and gain access to the system.
    - **Fleet Browsing:** Customers can explore the "Exclusive Fleet" to get a look at the models in detail and to know the daily rates.
    - **Reservation Management:** Customers can take a vehicle on rent for the dates they have chosen. The system then calculates the total cost of the rental based on the duration automatically. In addition, they can also see their own booking history and cancel those reservations which are still pending.
2. **Administrator (Admin):**
    - **Fleet Management:** Admins have an absolute command of vehicle inventory. In their hands, the car additions, the updating of the existing vehicle details (price, status, image) as well as the deleting of vehicles from the fleet are all possible.
    - **Booking Supervision:** Admins can have a look at all booking requests made by customers. It is their responsibility to approve or reject these bookings and to mark the cars as "Returned" once the rental is over.

Use Case Diagram

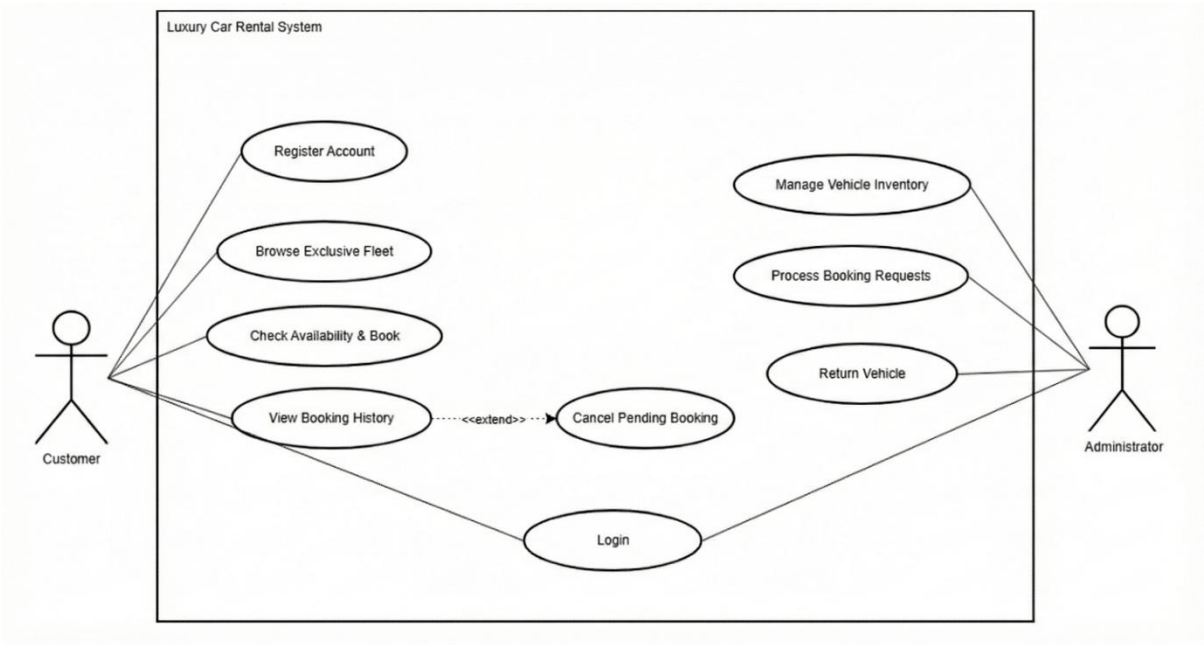

Figure 1 Use Case Diagram

Component diagram

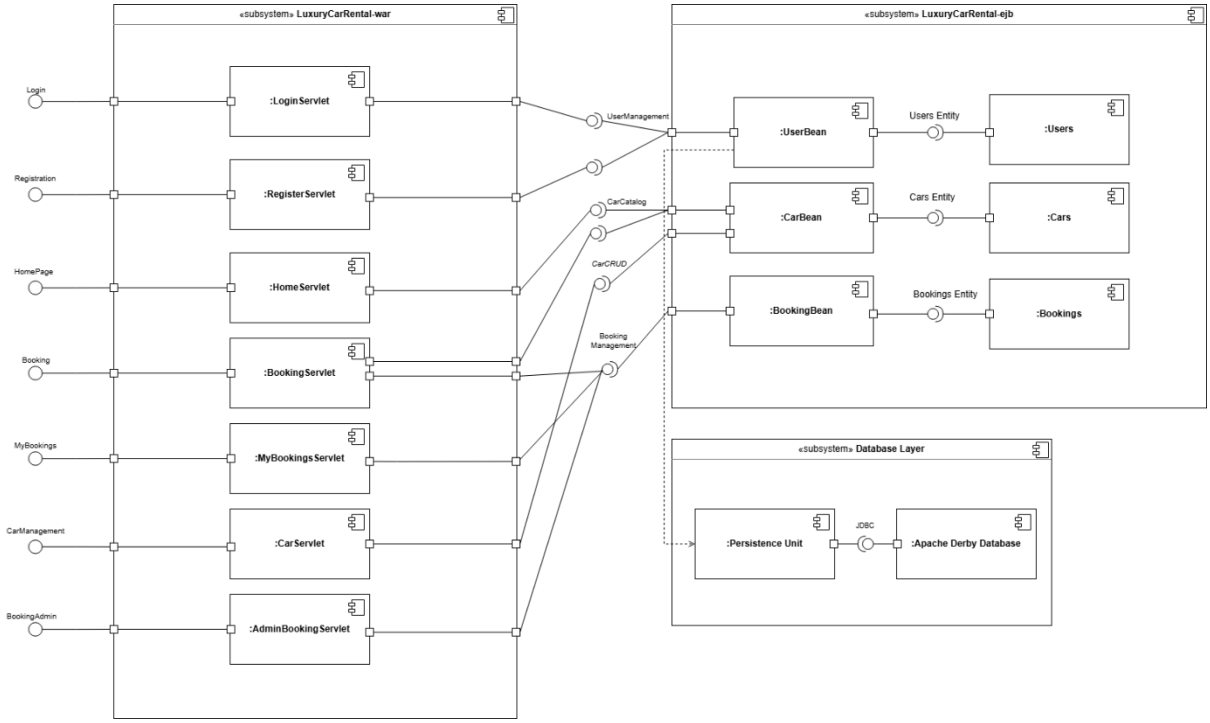

Figure 2 Component Diagram

Package Diagram

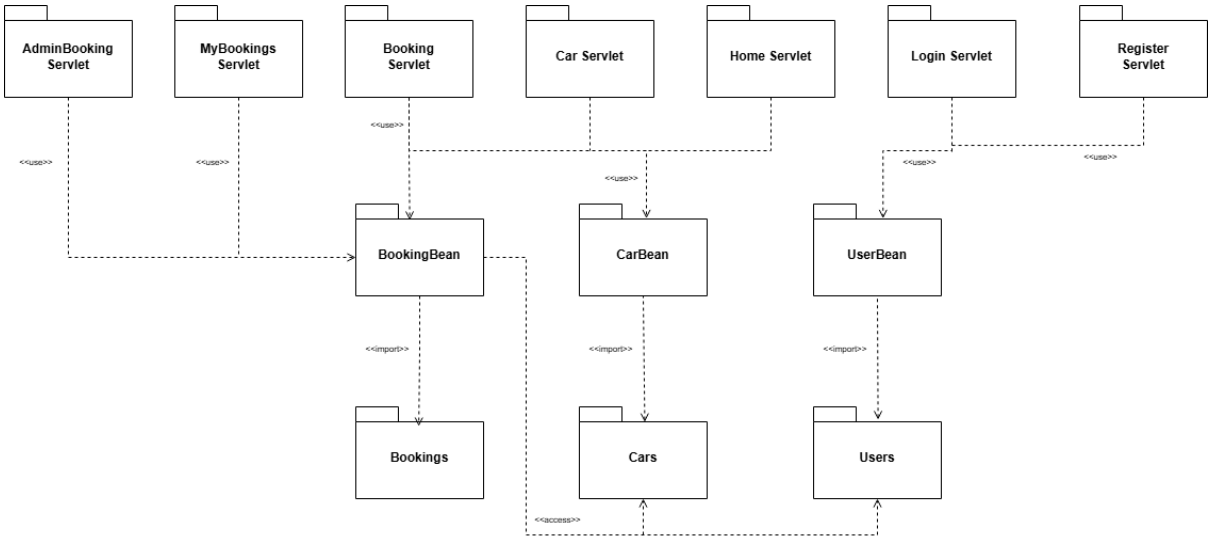

Figure 3 Package Diagram

Deployment diagram

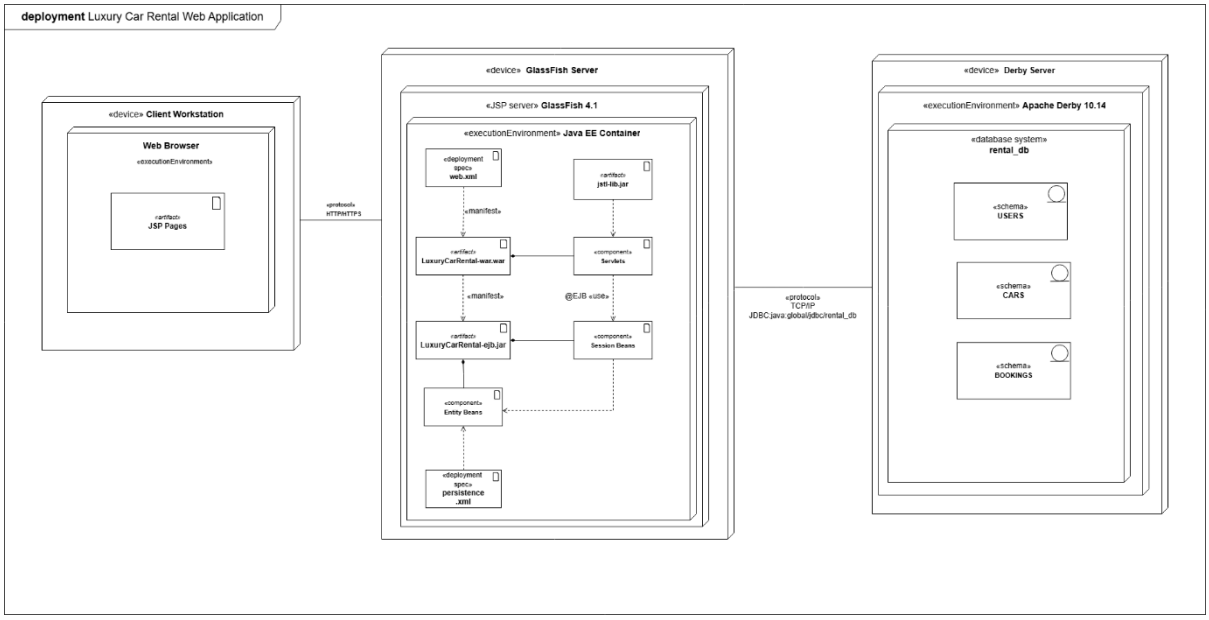

Figure 4 Deployment Diagram

Class diagram

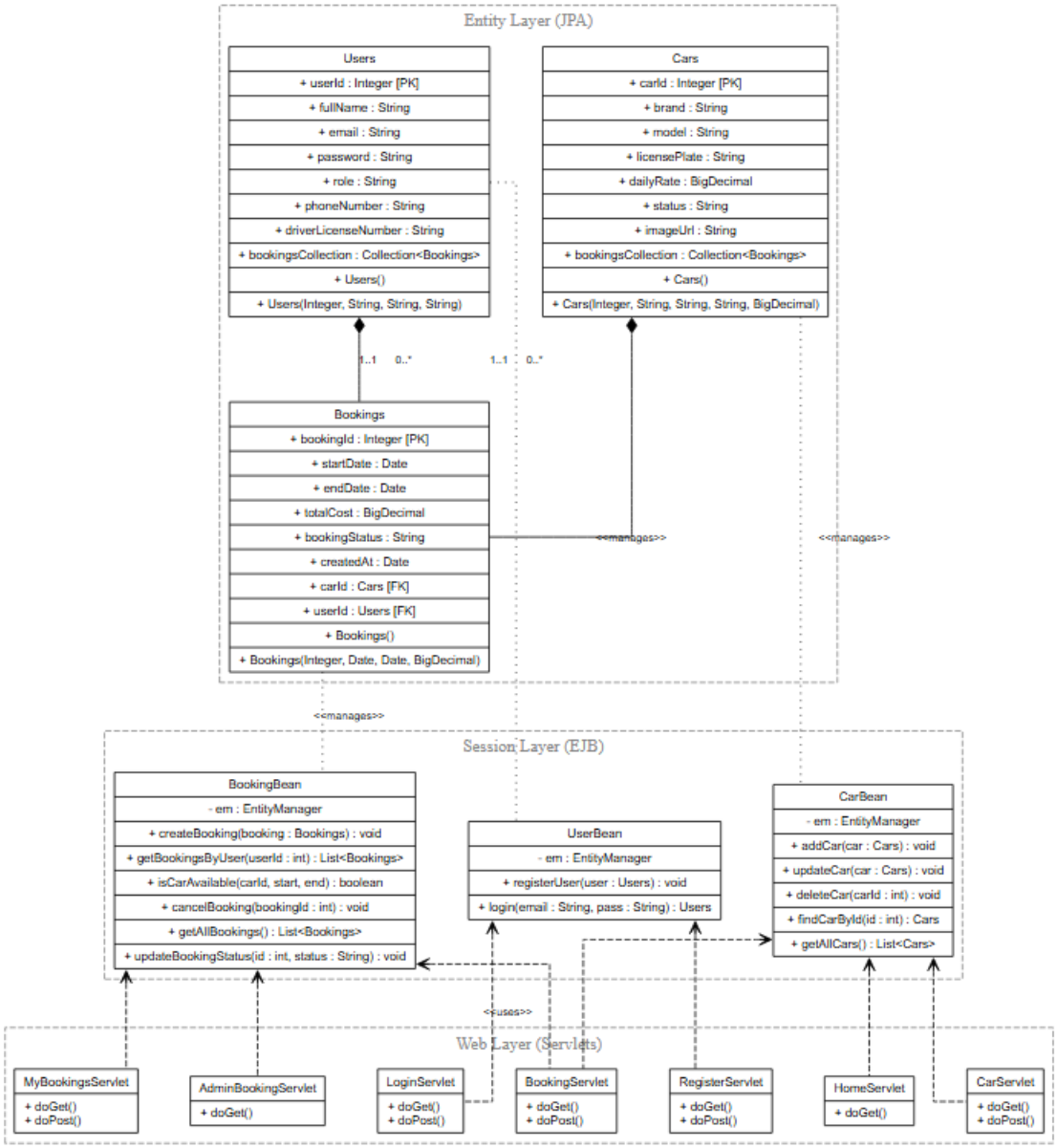

Figure 5 Class Diagram

2.2 Preparing Mockup

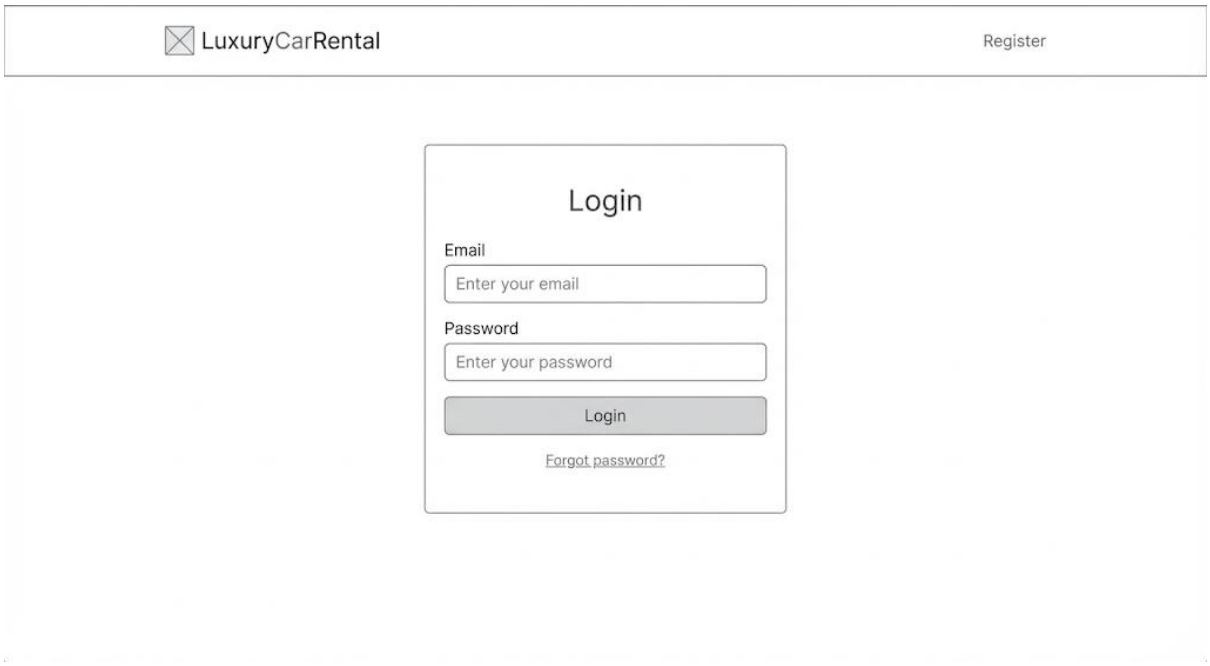

Figure 6 Login Page Mockup

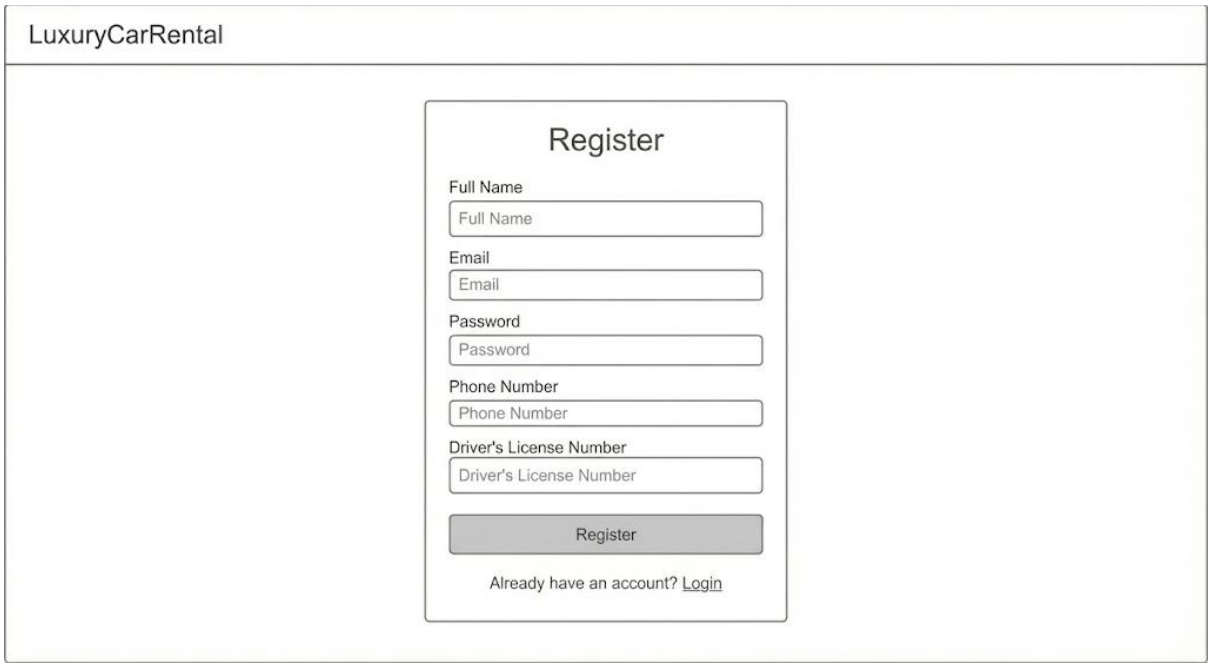

Figure 7 Registration Page Mockup

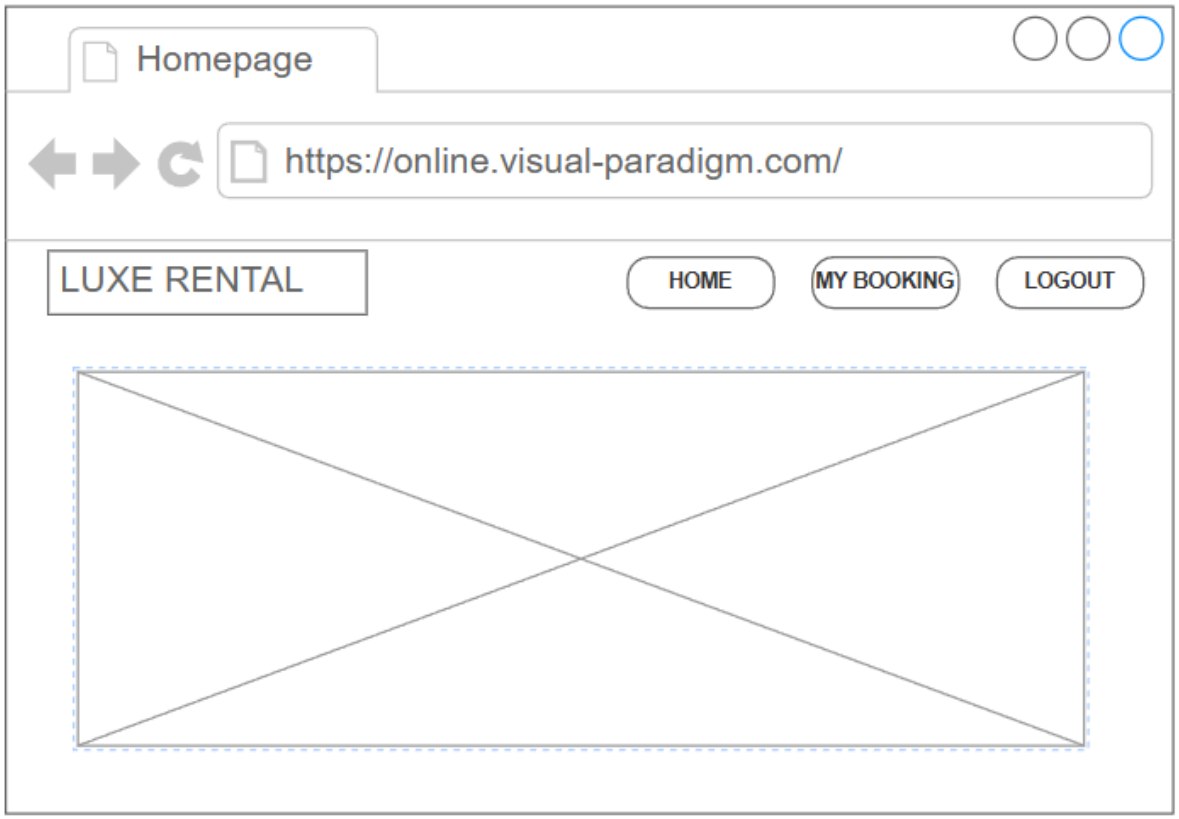

Figure 8 Homepage Mockup

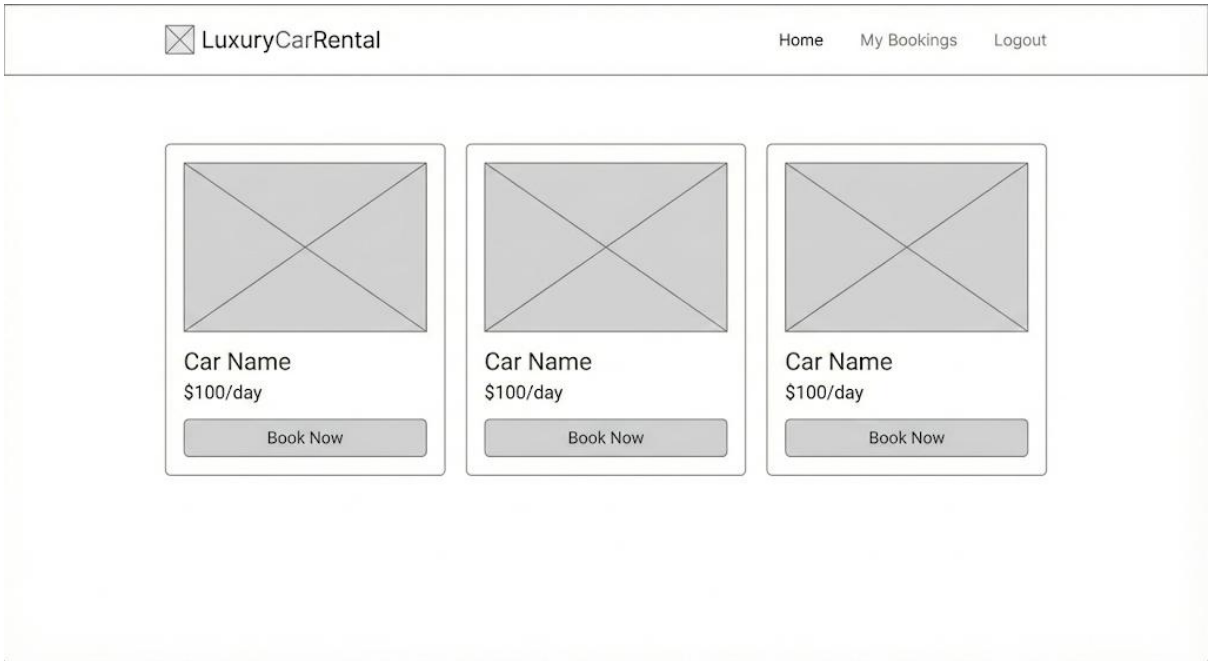

Figure 9 Menu Page Mockup

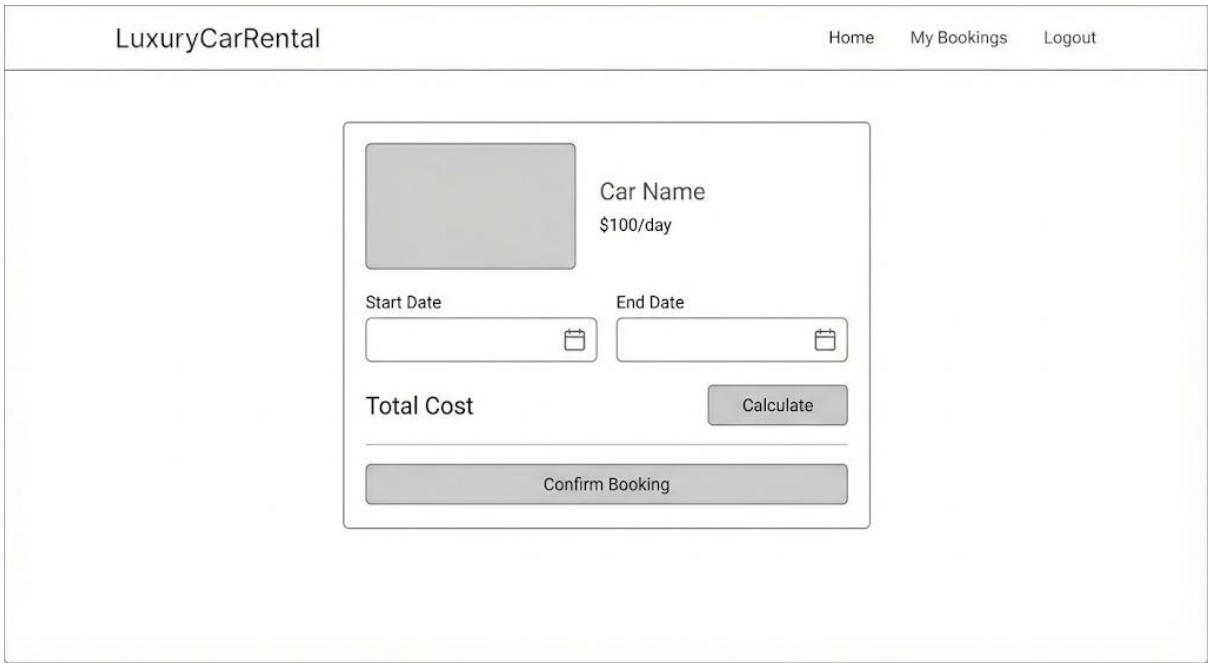

Figure 10 Booking Page Mockup

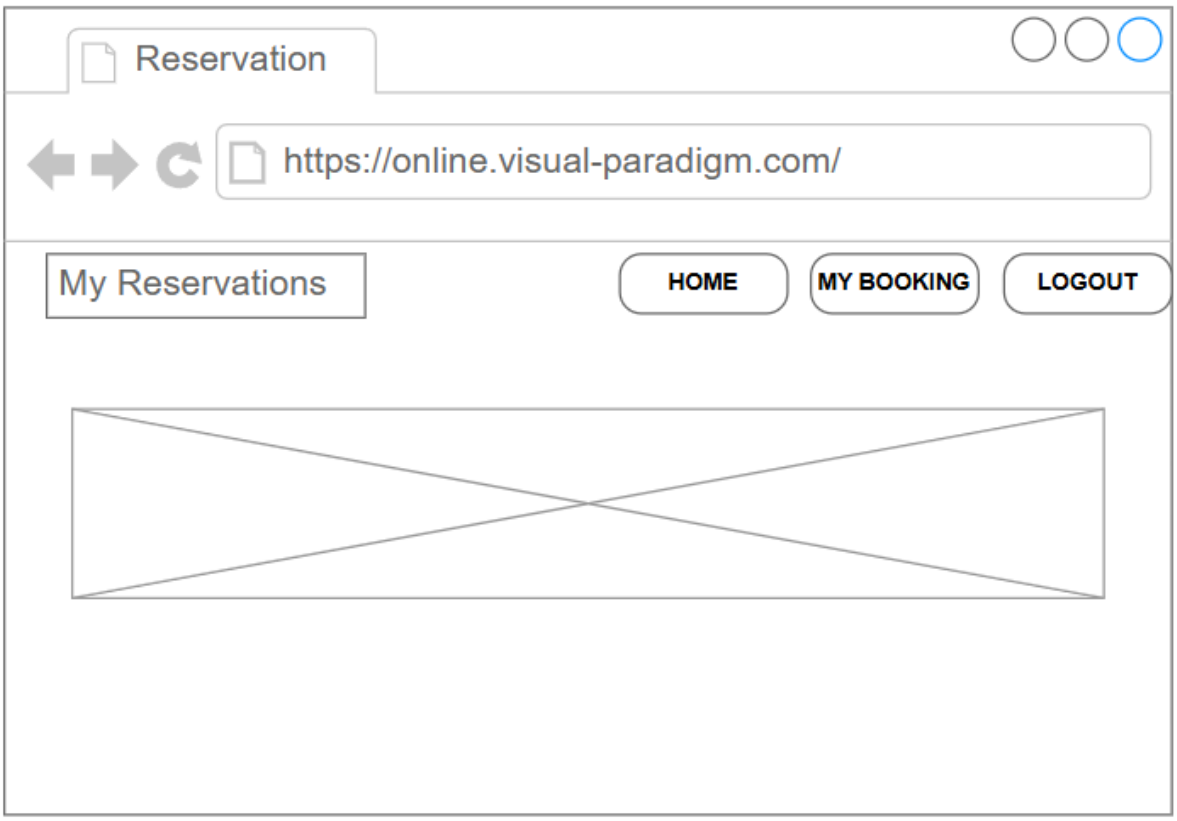

Figure 11 Reservation Details Mockup

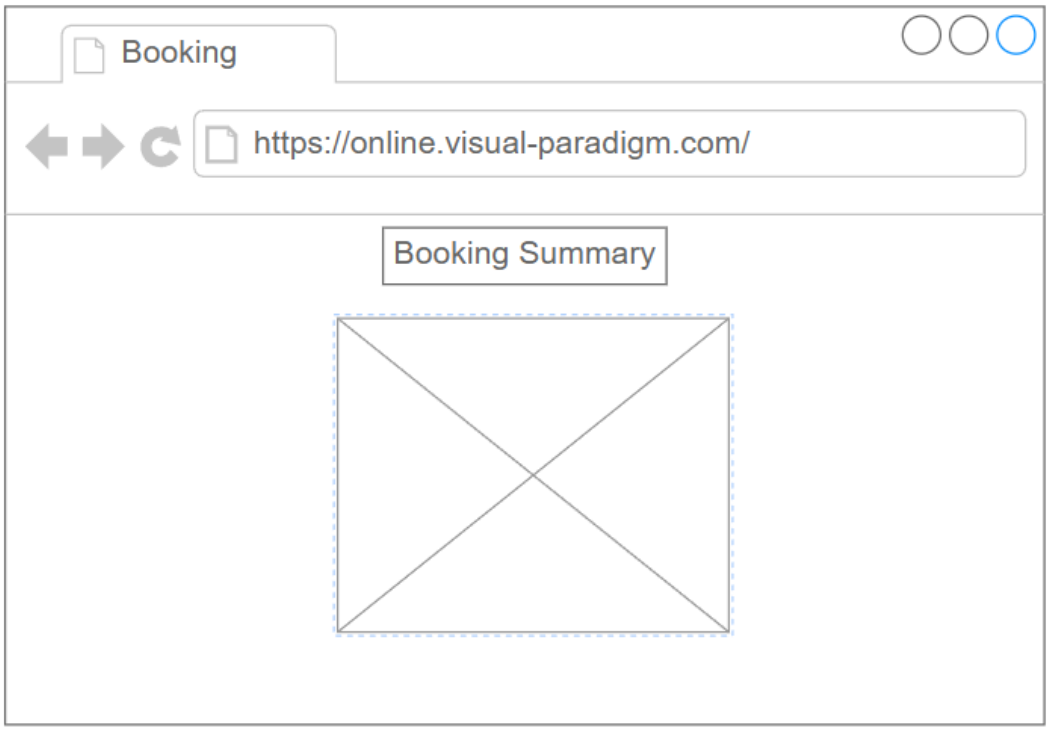

Figure 12Booking Summary Mockup

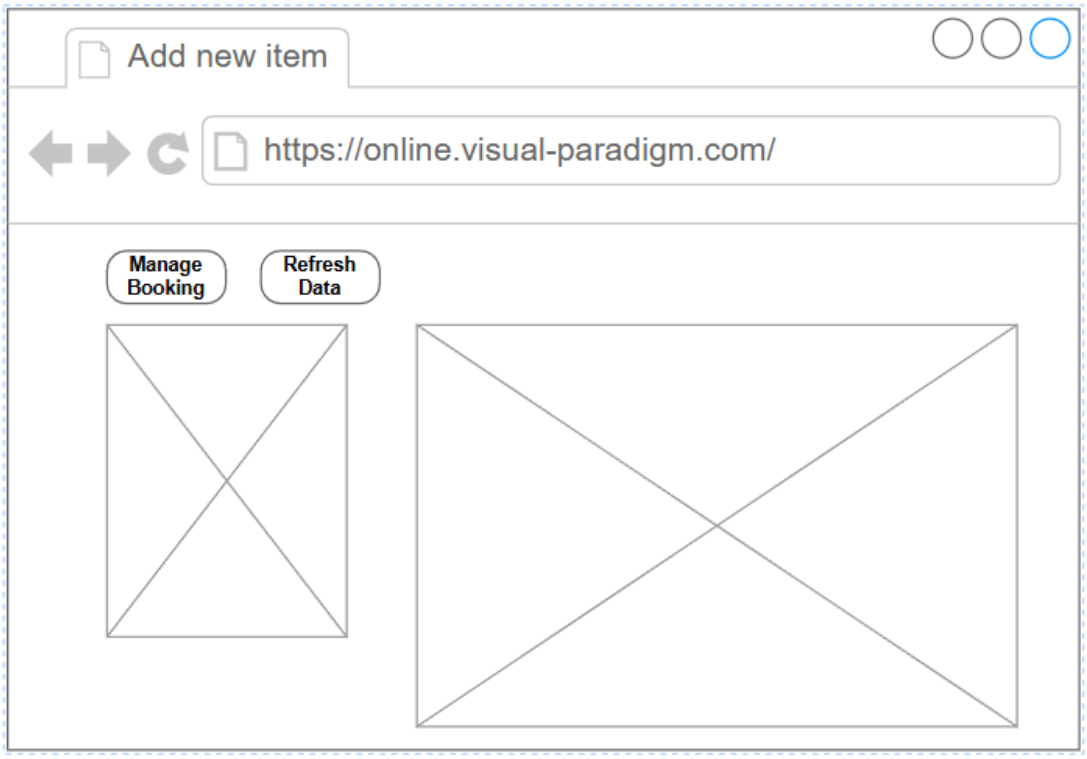

Figure 13 Add New Item Page Mockup

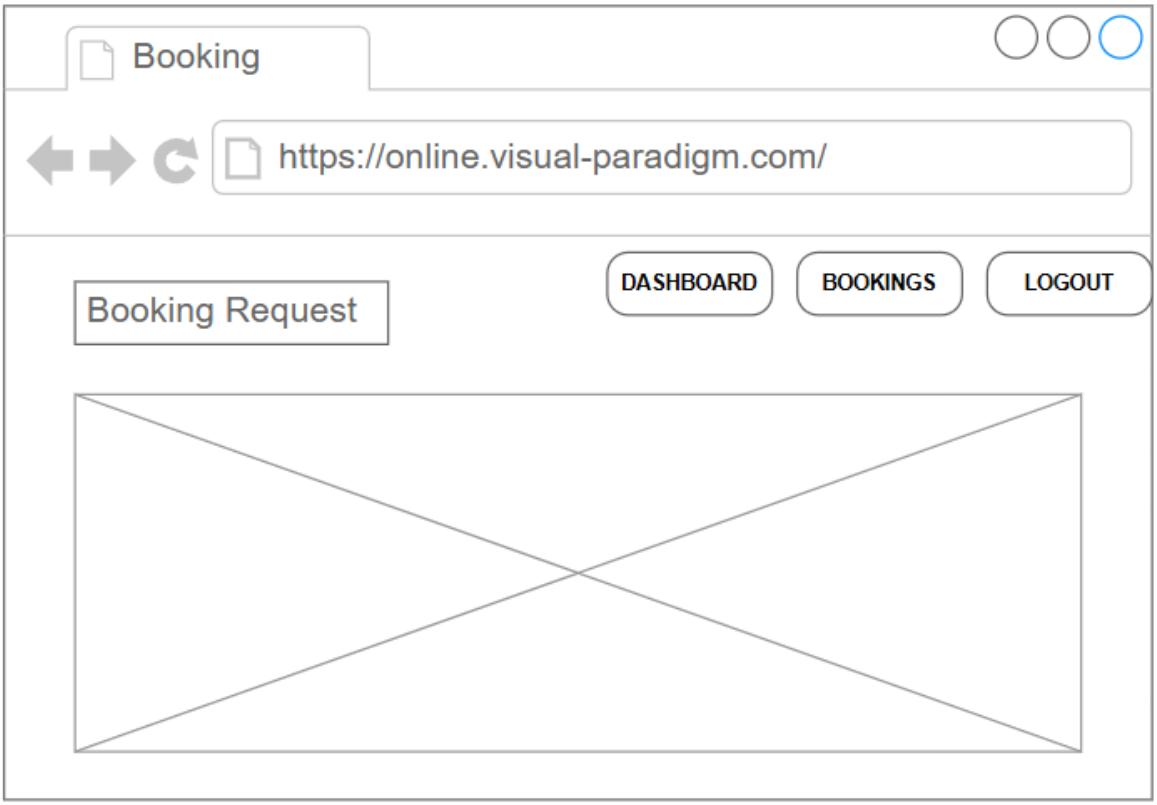

Figure 14 Booking Management Mockup

2.3 The Business Process Flow

Figure 15 Business Process Flow

2.4 Designing the Data Model

ER Diagram

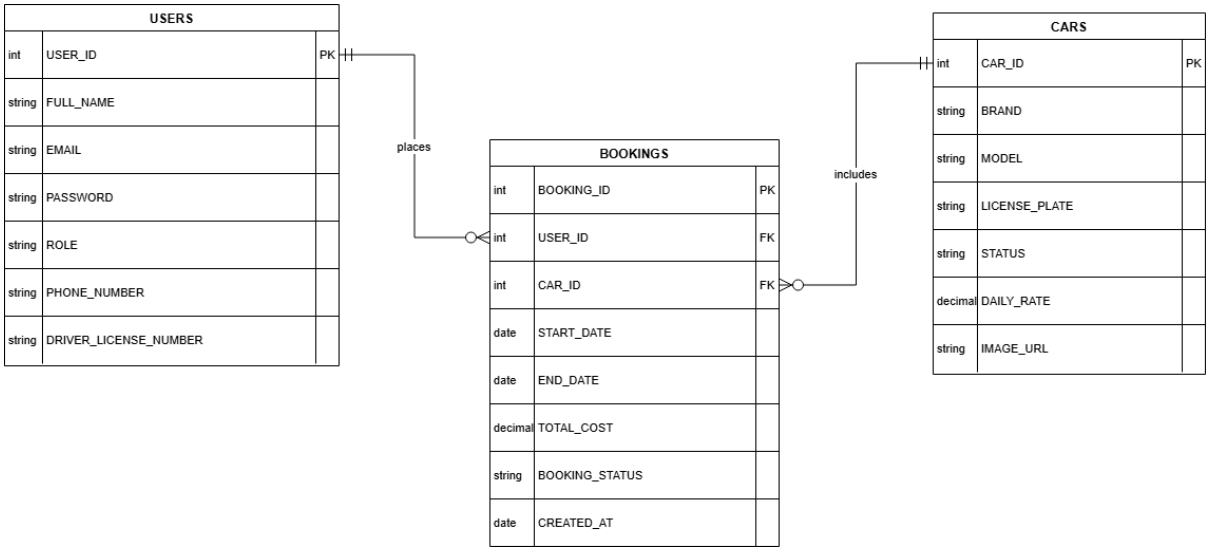

Figure 16 ER Diagram

Data Model

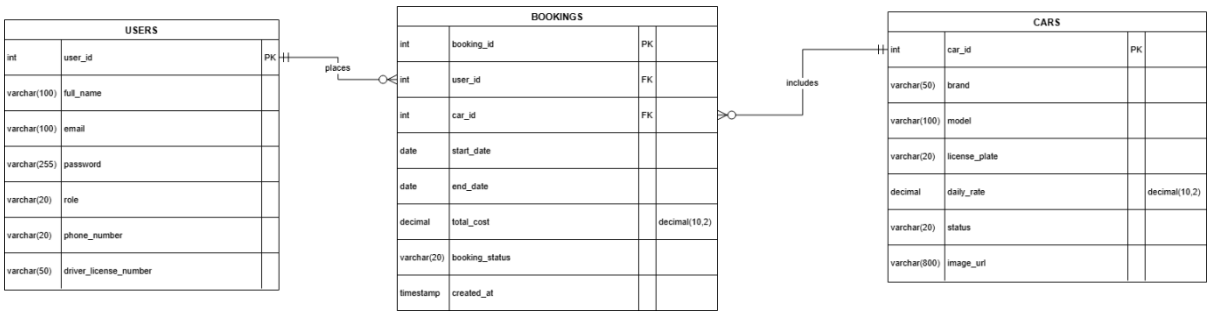

Figure 15 Data Model Diagram

3.0 Future Recommendation/Suggestion

The current version of the Luxury Car Rental System works well for basic tasks like managing users, controlling the fleet, and handling bookings. However, several changes are suggested to make the system better and ready for real-world business use. The following sections describe these improvements for functions, security, and user experience.

1. **Online Payment Integration**

   Currently, the system saves bookings as PENDING but does not process real payments. To make sure the business gets paid and to handle bookings automatically the system should use payment services like FPX or PayPal. This would let the system check payments instantly and change the booking status from PENDING to CONFIRMED without needing a person to do it.

2. **Automatic File Uploading**

   The current system asks admins to type in image filenames by hand which causes mistakes and takes too much time. Future versions should allow admins to upload files directly in the CarServlet using standard Java parts supported by NetBeans. This would let admins upload pictures from their computers and the system could save them automatically to the server storage.

3. **Visual Calendar for Bookings**

   To stop double bookings and keep data correct the system should use a visual calendar tool like Flatpickr. The current method forces users to type dates but a visual calendar can block out dates when cars are not available. This helps users see which cars they can rent right away before they try to book them.

4. **Better Search and Filters**

   As the number of cars grows looking through a long list becomes slow. The system should add search tools and filters like price range, brand, or availability. This would make it much faster and easier for customers to find exactly the car they want.

4.0 Conclusion

The development work of the Luxury Car Rental System has been an important learning milestone that has bridged the gap between theoretical knowledge and real software engineering. This project gave first-hand experience of creating a full-fledged web-based application using Java Enterprise Edition technologies. One of the main skills acquired was the ability to implement the Model View Controller architecture that separates the business logic from the user interface. Understanding of this structure helped in better structuring the form of coded differentiative where the Servlets are used for processing the requests and the Java Server Pages are used for a display layer.

Working with Enterprise JavaBeans was another important area of development. It showed how to encapsulate complex business rules such as calculating rental rates and availability of inventory in a dedicated layer. This resulted in a separation that made sure that the application was modular and easier to maintain. Furthermore, the project illustrated the significance of database management with the use of Entities. The mapping of Java objects to the database tables made it very easy to understand how the data should be persisted and backed up in a dynamic application environment.

The implementation of user authentication and session management gave deep insights on web security. Handling user roles requires careful handling of logic is in place to differentiate between administrators and customers to prevent unauthorized access to sensitive features such as the admin dashboard. Thus, highlighting the importance of secure coding practices to maintain the user's data vulnerability.

On the frontend side, working with the integration of the two: the HTML and CSS with the backend Java code, this allowed better understanding of how client-side and server-side technologies work together. For design varying scenarios like booking forms to dashboard tables, a responsive user interface designed for a positive user experience highlighted the need for a user-centric design approach. Additionally, addressing the real-life challenge of checking valid dates and replicating data inputs refined problem-solving skills and attention to detail.

Overall, this project effectively turns abstract ideas into a functional web system. It built a strong foundation in Full Stack Java Web development and set the stage in the fighting future with more complex enterprise-level challenges.

5.0 Appendix – Source Code Sample

1. CarBean.java

```
 1. package com.rental.session;
 2.
 3. import com.rental.entity.Cars;
 4. import java.util.List;
 5. import javax.ejb.Stateless;
 6. import javax.persistence.EntityManager;
 7. import javax.persistence.PersistenceContext;
 8.
 9. @Stateless
10. public class CarBean implements CarBeanLocal {
11.
12.
13.     @PersistenceContext(unitName = "LuxuryCarRental-ejbPU")
14.     private EntityManager em;
15.
16.     @Override
17.     public void addCar(Cars car) {
18.         em.persist(car);
19.     }
20.
21.     @Override
22.     public void updateCar(Cars car) {
23.         em.merge(car);
24.     }
25.
26.     @Override
27.     public void deleteCar(int carId) {
28.         Cars c = em.find(Cars.class, carId);
29.         if (c != null) {
30.             em.remove(c);
31.         }
32.     }
33.
34.     @Override
35.     public boolean isLicensePlateExists(String plate) {
36.     long count = (long) em.createQuery("SELECT COUNT(c) FROM Cars c WHERE c.licensePlate =
:plate")
37.             .setParameter("plate", plate)
38.             .getSingleResult();
39.     return count > 0;
40. }
41.
42.     @Override
43.     public Cars findCarById(int id) {
44.         return em.find(Cars.class, id);
45.     }
46.
47.     @Override
48.     public List<Cars> getAllCars() {
49.         // "Cars" here refers to the Class name, not the table name
50.         return em.createQuery("SELECT c FROM Cars c", Cars.class).getResultList();
51.     }
52.
53. }
54.
```

2. Cars.java

```
1. /*
2.  * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change
this license
3.  * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
4.  */
5. package com.rental.entity;
6.
7. import java.io.Serializable;
8. import java.math.BigDecimal;
9. import java.util.Collection;
10. import javax.persistence.Basic;
11. import javax.persistence.CascadeType;
12. import javax.persistence.Column;
13. import javax.persistence.Entity;
14. import javax.persistence.GeneratedValue;
15. import javax.persistence.GenerationType;
16. import javax.persistence.Id;
17. import javax.persistence.NamedQueries;
18. import javax.persistence.NamedQuery;
19. import javax.persistence.OneToMany;
20. import javax.persistence.Table;
21. import javax.validation.constraints.NotNull;
22. import javax.validation.constraints.Size;
23. import javax.xml.bind.annotation.XmlRootElement;
24. import javax.xml.bind.annotation.XmlTransient;
25.
26. /**
27.  *
28.  * @author khair
29.  */
30. @Entity
31. @Table(name = "CARS")
32. @XmlRootElement
33. @NamedQueries({
34.     @NamedQuery(name = "Cars.findAll", query = "SELECT c FROM Cars c"),
35.     @NamedQuery(name = "Cars.findByCarId", query = "SELECT c FROM Cars c WHERE c.carId =
:carId"),
36.     @NamedQuery(name = "Cars.findByBrand", query = "SELECT c FROM Cars c WHERE c.brand =
:brand"),
37.     @NamedQuery(name = "Cars.findByModel", query = "SELECT c FROM Cars c WHERE c.model =
:model"),
38.     @NamedQuery(name = "Cars.findByLicensePlate", query = "SELECT c FROM Cars c WHERE
c.licensePlate = :licensePlate"),
39.     @NamedQuery(name = "Cars.findByDailyRate", query = "SELECT c FROM Cars c WHERE
c.dailyRate = :dailyRate"),
40.     @NamedQuery(name = "Cars.findByStatus", query = "SELECT c FROM Cars c WHERE c.status =
:status"),
41.     @NamedQuery(name = "Cars.findByImageUrl", query = "SELECT c FROM Cars c WHERE
c.imageUrl = :imageUrl")})
42.
43. public class Cars implements Serializable {
44.
45.     private static final long serialVersionUID = 1L;
46.     @Id
47.     @GeneratedValue(strategy = GenerationType.IDENTITY)
48.     @Basic(optional = false)
49.     @Column(name = "CAR_ID")
50.     private Integer carId;
51.     @Basic(optional = false)
52.     @NotNull
53.     @Size(min = 1, max = 50)
54.     @Column(name = "BRAND")
55.     private String brand;
56.     @Basic(optional = false)
57.     @NotNull
58.     @Size(min = 1, max = 50)
59.     @Column(name = "MODEL")
```

```
60.     private String model;
61.     @Basic(optional = false)
62.     @NotNull
63.     @Size(min = 1, max = 20)
64.     @Column(name = "LICENSE_PLATE")
65.     private String licensePlate;
66.     @Basic(optional = false)
67.     @NotNull
68.     @Column(name = "DAILY_RATE")
69.     private BigDecimal dailyRate;
70.     @Size(max = 20)
71.     @Column(name = "STATUS")
72.     private String status;
73.     @Size(max = 255)
74.     @Column(name = "IMAGE_URL")
75.     private String imageUrl;
76.     @OneToMany(cascade = CascadeType.ALL, mappedBy = "carId")
77.     private Collection<Bookings> bookingsCollection;
78.
79.     public Cars() {
80.     }
81.
82.     public Cars(Integer carId) {
83.         this.carId = carId;
84.     }
85.
86.     public Cars(Integer carId, String brand, String model, String licensePlate, BigDecimal
dailyRate) {
87.         this.carId = carId;
88.         this.brand = brand;
89.         this.model = model;
90.         this.licensePlate = licensePlate;
91.         this.dailyRate = dailyRate;
92.     }
93.
94.     public Integer getCarId() {
95.         return carId;
96.     }
97.
98.     public void setCarId(Integer carId) {
99.         this.carId = carId;
100.     }
101.
102.     public String getBrand() {
103.         return brand;
104.     }
105.
106.     public void setBrand(String brand) {
107.         this.brand = brand;
108.     }
109.
110.     public String getModel() {
111.         return model;
112.     }
113.
114.     public void setModel(String model) {
115.         this.model = model;
116.     }
117.
118.     public String getLicensePlate() {
119.         return licensePlate;
120.     }
121.
122.     public void setLicensePlate(String licensePlate) {
123.         this.licensePlate = licensePlate;
124.     }
125.
126.     public BigDecimal getDailyRate() {
127.         return dailyRate;
128.     }
```

```
129.
130.     public void setDailyRate(BigDecimal dailyRate) {
131.         this.dailyRate = dailyRate;
132.     }
133.
134.     public String getStatus() {
135.         return status;
136.     }
137.
138.     public void setStatus(String status) {
139.         this.status = status;
140.     }
141.
142.     public String getImageUrl() {
143.         return imageUrl;
144.     }
145.
146.     public void setImageUrl(String imageUrl) {
147.         this.imageUrl = imageUrl;
148.     }
149.
150.     @XmlTransient
151.     public Collection<Bookings> getBookingsCollection() {
152.         return bookingsCollection;
153.     }
154.
155.     public void setBookingsCollection(Collection<Bookings> bookingsCollection) {
156.         this.bookingsCollection = bookingsCollection;
157.     }
158.
159.     @Override
160.     public int hashCode() {
161.         int hash = 0;
162.         hash += (carId != null ? carId.hashCode() : 0);
163.         return hash;
164.     }
165.
166.     @Override
167.     public boolean equals(Object object) {
168.         // TODO: Warning - this method won't work in the case the id fields are not set
169.         if (!(object instanceof Cars)) {
170.             return false;
171.         }
172.         Cars other = (Cars) object;
173.         if ((this.carId == null && other.carId != null) || (this.carId != null &&
!this.carId.equals(other.carId))) {
174.             return false;
175.         }
176.         return true;
177.     }
178.
179.     @Override
180.     public String toString() {
181.         return "com.rental.entity.Cars[ carId=" + carId + " ]";
182.     }
183.
184. }
185.
```

3. CarServlet.java

```java
package com.rental.web;

import com.rental.entity.Cars;
import com.rental.entity.Users;
import com.rental.session.CarBeanLocal;
import java.io.IOException;
import java.math.BigDecimal;
import java.util.List;
import javax.ejb.EJB;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;

@WebServlet(name = "CarServlet", urlPatterns = {"/CarServlet"})
public class CarServlet extends HttpServlet {

    @EJB
    private CarBeanLocal carBean;

    // doGet handles: Listing Cars, Deleting Cars, and LOADING the Edit Page
    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        HttpSession session = request.getSession();
        Users user = (Users) session.getAttribute("user");

        // Security: If not admin, go to login
        if (user == null || !"ADMIN".equals(user.getRole())) {
            response.sendRedirect("login.jsp");
            return;
        }

        String action = request.getParameter("action");

        if ("delete".equals(action)) {
            // Case 1: Delete a car
            int id = Integer.parseInt(request.getParameter("id"));
            carBean.deleteCar(id);
            // Refresh the list by calling this same servlet again
            response.sendRedirect("CarServlet");

        } else if ("edit".equals(action)) {
            // Case 2: Load the Edit Form
            int id = Integer.parseInt(request.getParameter("id"));
            Cars c = carBean.findCarById(id);
            request.setAttribute("car", c);
            request.getRequestDispatcher("edit_car.jsp").forward(request, response);

        } else {
            // Case 3 (Default): Show the Dashboard with the List
            List<Cars> list = carBean.getAllCars();
            request.setAttribute("carList", list);
            request.getRequestDispatcher("admin_dashboard.jsp").forward(request, response);
        }
    }

    // doPost handles: Adding New Cars AND Updating Existing Cars
    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        request.setCharacterEncoding("UTF-8");
        String action = request.getParameter("action"); // Check if it's "update"
```

```java

        try {
            // Common Fields
            String brand = request.getParameter("brand");
            String model = request.getParameter("model");
            String license = request.getParameter("license");
            BigDecimal rate = new BigDecimal(request.getParameter("rate"));
            String image = request.getParameter("image");

            if ("update".equals(action)) {
                // --- UPDATE LOGIC ---
                int id = Integer.parseInt(request.getParameter("id"));
                String status = request.getParameter("status");

                Cars c = new Cars();
                c.setCarId(id);
                c.setBrand(brand);
                c.setModel(model);
                c.setLicensePlate(license);
                c.setDailyRate(rate);
                c.setImageUrl(image);
                c.setStatus(status);

                carBean.updateCar(c);

            } else {
                // --- ADD NEW CAR LOGIC ---
                // [NEW CODE] Check if license exists before adding
                if (carBean.isLicensePlateExists(license)) {
                    // 1. Set error message
                    request.setAttribute("errorMessage", "Error: License Plate already exists!");

                    // 2. Reload the list so the dashboard isn't empty
                    List<Cars> list = carBean.getAllCars();
                    request.setAttribute("carList", list);

                    // 3. Forward back to dashboard (don't redirect, or you lose the error message)
                    request.getRequestDispatcher("admin_dashboard.jsp").forward(request, response);
                    return; // STOP here so we don't redirect below
                }
                Cars c = new Cars();
                c.setBrand(brand);
                c.setModel(model);
                c.setLicensePlate(license);
                c.setDailyRate(rate);
                c.setImageUrl(image);
                c.setStatus("AVAILABLE");

                carBean.addCar(c);
            }

            response.sendRedirect("admin_dashboard.jsp");

        } catch (Exception e) {
            e.printStackTrace();
            response.sendRedirect("admin_dashboard.jsp?error=true");
        }
    }
}

```

6.0 References

1. GeeksforGeeks. (2024). *Introduction to Java Servlets*. https://www.geeksforgeeks.org/introduction-java-servlets/
2. W3Schools. (2024). *Java JDBC Tutorial*. https://www.w3schools.com/java/java_mysql_connect.asp
3. W3Schools. (2024). *CSS Tutorial*.

   https://www.w3schools.com/css/

4. UML-Diagrams.org. (2024). *Unified Modeling Language (UML) description*. https://www.uml-diagrams.org/
5. Visual Paradigm. (n.d.). *What is Unified Modeling Language (UML)?* https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-uml/